

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Лабораторная работа №1
по дисциплине
«Методы тестирования программного обеспечения»

Студент,

группы 5130201/30101

_____ Мелещенко С. И.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Методы ручного тестирования	5
3 Технология инспекции кода	8
3.1 Состав группы	8
3.2 Исходная программа	8
3.3 Список вопросов и ответов на них	13
4 Код программы с внесенными изменениями	20
5 Обобщение работы	27
Заключение	28
Список литературы	29

Введение

Тестирование — это систематизированный анализ каждой составной части ПО. Оно необходимо для вскрытия дефектов, ошибок и несоответствий требованиям технического задания. Тестирование играет решающую роль в разработке, поскольку определяет качество продукта и его работоспособность в реальных условиях.

В большинстве случаев программисты неспособны эффективно тестировать собственные программы, поскольку им трудно психологически перестроиться на демонстрацию собственных огрехов.

Целесообразность вычитывания кода зависит от ряда факторов: масштаб и сложность приложения, численный состав команды разработчиков, временной график проекта (например, щадящий он или напряженный) и, конечно же, квалификация и профессиональная культура команды программистов.

Рассмотрим процесс тестирования, не предполагающий использования компьютера и осуществляемый непосредственно человеком, — ручное тестирование. Оно зарекомендовало себя достаточно эффективным методом обнаружения ошибок, и его разновидности следует задействовать в любом программном проекте. Методы ручного тестирования должны применяться к написанному исходному коду до того, как начнется его тестирование на компьютере. Можно также самостоятельно создавать аналогичные методики и применять их на более ранних стадиях процесса программирования (например, в конце каждого этапа разработки проекта). Во-первых, общепризнан тот факт, что чем раньше ошибки будут обнаружены, тем дешевле обойдется их устранение и тем выше вероятность того, что исправления будут корректными. Во-вторых, замечено, что у программиста, тестирующего программное обеспечение на компьютере, определенным образом меняются психологические установки. В нем быстро нарастает внутреннее напряжение и возникает непреодолимое желание как можно скорее разделаться со злополучной ошибкой. Как следствие, при исправлении ошибок, обнаруженных в процессе тестирования программы на компьютере, программист допускает больше просчетов, чем при исправлении ошибок, обнаруженных на более ранних этапах. Дополнительным преимуществом сквозных просмотров, снижающим стоимость отладки (исправления ошибок), является возможность точной локализации ошибки. Кроме того, в процессе сквозного просмотра обычно удается выявить целую группу ошибок, которые впоследствии можно устранить все вместе. При тестировании программы на компьютере обычно проявляются лишь признаки ошибок (например, программа не может корректно завершиться или выводит бессмысленные результаты), а сами они обнаруживаются и устраняются по отдельности.

1 Постановка задачи

В данной лабораторной работе требуется провести инспекцию кода, выступая в роли разработчика программы.

Для выполнения данной задачи необходимо:

- 1) изучить методы ручного тестирования;
- 2) провести инспекцию кода, проанализировать ее результаты;
- 3) исправить программу в соответствии с рекомендациями специалиста по тестированию.

2 Методы ручного тестирования

Существуют три основных метода ручного тестирования:

- 1) инспекция кода;
- 2) сквозной просмотр;
- 3) тестирование удобства использования.

Инспекция кода

Инспекция кода — это набор процедур и методик обнаружения ошибок путем анализа (чтения) кода группой специалистов. В большинстве случаев, говоря об инспекции кода, основное внимание уделяют процедурным вопросам, формам, подлежащим заполнению, и т.п. Мы же, после предварительного ознакомления с общей концепцией, сконцентрируемся на изучении самих методов обнаружения ошибок.

Группа инспектирования кода

Обычно в состав группы входят четыре человека, один из которых играет роль координатора. Координатор должен быть квалифицированным программистом, но не автором тестируемой программы, детальное знание которой от него не требуется. Вторым участником группы является программист, а остальными — проектировщик программы (если это не сам программист) и специалист по тестированию.

В рамках выполнения лабораторной работы в инспекции кода участвовало всего три человека: программист, специалист по тестированию и координатор.

Сквозной просмотр Как и инспекция, сквозной просмотр кода представляет собой совокупность процедур и методов обнаружения ошибок в коде путем его просмотра участниками группы тестирования. Такой просмотр имеет много общего с процессом инспектирования, но при этом используются несколько иные процедуры и методы. Подобно инспекции, сквозной просмотр проводится в форме непрерывного заседания, длящегося не более двух часов. Число участников группы, выполняющей сквозной просмотр, составляет 3-5 человек. Один из них выполняет функции, аналогичные функциям координатора в группе инспектирования. Еще один человек играет роль секретаря и записывает все найденные ошибки, а третий участник — тестировщик. Мнения о том, кто должен входить в группу и стоит ли ее расширять до пяти человек, расходятся. Конечно, одним из этих людей должен быть программист, написавший программу. К числу других возможных кандидатур относятся:

- 1) эксперт по используемому языку программирования;
- 2) начинающий программист (на точку зрения которого не повлияет предыдущий опыт);
- 3) человек, который в конечном счете будет обслуживать программу;
- 4) участник какого-либо другого проекта;
- 5) кто-нибудь из той же группы программистов, что и автор программы.

За несколько дней до заседания необходимые материалы раздают участникам, чтобы они могли заблаговременно ознакомиться с проверяемым программным обеспечением. Однако работа самого заседания организуется иначе. Вместо простого чтения кода программы или ее проверки с помощью контрольного списка ошибок участники заседания “играют в компьютер”. Тот, кто был назначен тестировщиком, прихо-

дит на совещание с предварительно записанными на листах бумаги тестами — представительными наборами входных (и ожидаемых выходных) данных для программы или модуля. Во время заседания участники “прогоняют в уме” каждый тест, т.е. подвергают тестовые данные обработке вручную в соответствии с логикой программы. Состояние программы (значения переменных) отслеживается на бумаге или на доске. Конечно, тестов должно быть не слишком много, и они должны быть простыми по своей природе, потому что скорость ручного выполнения запрограммированных действий на много порядков меньше, чем скорость вычислений на компьютере. Следовательно, тесты сами по себе не играют критической роли; скорее они служат средством “разогрева” участников и основой для постановки вопросов программисту, касающихся логики проектирования и принятых допущений. В большинстве сквозных просмотров при выполнении самих тестов находят меньше ошибок, чем при опросе программиста.

Проверка за столом

Третьим методом ручного обнаружения ошибок является давно известная “проверка за столом”. Она может рассматриваться как инспекция или сквозной просмотр кода, выполняемые одним человеком, который вычитывает код программы, проверяет его, руководствуясь контрольным списком ошибок, и (или) прогоняет через логику программы тестовые данные. Для большинства людей проверка за столом является относительно непродуктивной. Это объясняется прежде всего тем, что такая проверка представляет собой полностью неупорядоченный процесс. Вторая, более важная причина заключается в том, что проверка за столом вступает в противоречие со вторым принципом тестирования, согласно которому тестирование программистом собственных программ обычно оказывается неэффективным. Следовательно, оптимальный вариант состоит в том, чтобы такую проверку выполнял человек, не являющийся автором программы (например, два программиста могут обмениваться программами для взаимной проверки, а не проверять собственные программы), но даже в этом случае такая проверка менее эффективна, чем сквозные просмотры или инспекции. В основном именно по этой причине лучше, чтобы сквозные просмотры или инспекции осуществлялись в группе. Совещательный групповой метод благоприятствует созданию атмосферы здоровой конкуренции, поскольку все участники хотят показать себя с наилучшей стороны при нахождении ошибок. При проверке за столом подобный безусловно ценный эффект отсутствует. Короче говоря, проверка за столом, конечно, полезна, но она гораздо менее эффективна, чем инспекция или сквозной просмотр кода.

Рецензирование

Последний из рассматриваемых нами процессов экспертного просмотра программ человеком не имеет прямого отношения к тестированию программного обеспечения (т.е. его целью не является обнаружение дефектов). И все же описание этого процесса приводится здесь, поскольку он связан с идеей чтения программного кода. Рецензирование — это процедура анонимной оценки общих характеристик качества, обслуживаемости, расширяемости, удобства использования и ясности программного обеспечения. Цель данного метода — предоставить программисту возможность получить стороннюю оценку результатов своего труда. Общее руководство процессом осуществляет администратор, выбираемый из числа программистов. В свою очередь, администратор отбирает в группу рецензентов от 6 до 20 участников (6 - это необходимый минимум, обеспечивающий анонимность оценок). Предполагается, что все

участники специализируются в одной области (например, не следует включать в одну группу разработчиков Java-приложений и системных программистов, пишущих на языке ассемблера). Каждый из участников предоставляет для рецензирования две своих программы, одну из которых он считает наилучшей, а вторую — наихудшей по качеству. Когда будут собраны все программы, их распределяют случайным образом между участниками. Каждому участнику дают для рецензирования четыре программы. Две из них относятся к категории “наилучших”, а две — к категории “наихудших” программ, но рецензенту не сообщают, какой именно является каждая из них. Любой участник тратит на просмотр одной программы 30 минут и заполняет ее оценочную анкету. После просмотра всех четырех программ рецензент оценивает их относительное качество. Анкета предлагает оценивать программы по десятибалльной шкале (“1” означает твердое “да”, “10” — твердое “нет”), отвечая, например, на следующие вопросы.

- 1) Легко ли было разобраться в программе?
- 2) Хорошо ли различимы и достаточно ли разумны используемые в программе концепции высокоуровневого проектирования?
- 3) Хорошо ли различимы и достаточно ли разумны используемые в программе концепции низкоуровневого проектирования?
- 4) Легко ли вам было бы модифицировать данную программу?
- 5) Испытывали бы вы чувство удовлетворения, написав такую программу?

Рецензента также просят предоставить свои замечания к программе и дать рекомендации по ее улучшению. После просмотра всех программ каждому участнику передают анкеты с оценками двух его программ. Кроме того, участники получают статистическую сводку, отражающую общие и детализированные данные о рейтинге их собственных программ среди всего набора, а также анализ того, насколько оценки, данные участником чужим программам, близки к оценкам тех же программ со стороны других рецензентов.

3 Технология инспекции кода

3.1 Состав группы

Заседание происходило в следующем составе:

- 1) секретарь Мартыненко Анна, исполняющая роль координатора;
- 2) специалист по тестированию Михалец Мартин;
- 3) программист и проектировщик Мелещенко Софья.

3.2 Исходная программа

Название: Калькулятор для умножения матриц с измерением и выводом производительности.

Дано.

1. Консольный ввод пользователя.
2. Четыре положительных целых числа, задающих размеры двух матриц:
 - Количество строк первой матрицы.
 - Количество столбцов первой матрицы.
 - Количество строк второй матрицы.
 - Количество столбцов второй матрицы.
3. Программа автоматически заполняет матрицы случайными значениями типа `double`.

Требуется.

1. Проверить возможность умножения матриц (количество столбцов первой матрицы должно быть равно количеству строк второй).
2. Выполнить умножение матриц стандартным методом (тройным вложенным циклом) и измерить время выполнения (10 прогонов).
3. Выполнить умножение матриц оптимизированным методом (блочным умножением с размером блока 64) и измерить время выполнения (10 прогонов).
4. Вывести среднее время выполнения для каждого метода.

Ограничения.

1. Размеры матриц должны быть положительными целыми числами.
2. Умножение возможно только если количество столбцов первой матрицы равно количеству строк второй.

3. Матрицы заполняются случайными числами в диапазоне $[0, 1)$.

4. Время измеряется в миллисекундах.

Таблица 1

Спецификация

Входные данные	Ожидаемый результат	Комментарий
2, 2, 2, 2	Успешное умножение. Вывод времени выполнения для обоих методов.	Минимальные квадратные матрицы (2x2).
3, 4, 4, 5	Успешное умножение. Вывод времени выполнения.	Прямоугольные матрицы (3x4 и 4x5).
3, 2, 3, 2	Ошибка: "Матрицы такой размерности нельзя перемножить!"	Несовместимые размеры (3x2 и 3x2).
0, 5, 5, 5	Ошибка: "Ошибка! Введите положительное число!"	Некорректный размер (ноль строк).
5, -2, 5, 5	Ошибка: "Ошибка! Введите положительное число!"	Некорректный размер (отрицательные столбцы).
abc, 5, 5, 5	Ошибка: "Ошибка! Введите целое число!"	Нечисловой ввод для размера.
5, 5, 5.5, 5	Ошибка: "Ошибка! Введите целое число!"	Дробный ввод для размера.
1, 1000, 1000, 1	Успешное умножение. Результат - матрица 1x1 (скалярное произведение).	Матрица-строка и матрица-столбец.
50, 100, 100, 50	Успешное умножение. Вывод среднего времени за 10 прогонов для каждого метода.	Типичный случай умножения прямоугольных матриц.

Входные данные (размеры)	Ожидаемый результат	Комментарий
200, 200, 200, 200	Успешное умножение. Явное преимущество блочного метода по времени.	Большие матрицы для демонстрации оптимизации.

Код исходного файла программы представлен в листинге ниже.

Листинг 1. Код программы

```

1 import java.util.Scanner;
2
3 public class MatrixMult {
4     public static double[][] multiply(double[][] firstMatrix,
5         double[][] secondMatrix) {
6         int firstRows = firstMatrix.length;
7         int firstColumns = firstMatrix[0].length;
8         int secondRows = secondMatrix.length;
9         int secondColumns = secondMatrix[0].length;
10        double[][] result = new double[firstRows][secondColumns];
11        for (int i = 0; i < firstRows; i++) {
12            for (int j = 0; j < firstColumns; j++) {
13                for (int k = 0; k < secondColumns; k++) {
14                    result[i][k] += firstMatrix[i][j] *
15                        secondMatrix[j][k];
16                }
17            }
18        }
19        return result;
20    }
21
22    public static int readPositiveInt(Scanner scanner, String
23        message) {
24        while (true) {
25            System.out.print(message);
26            if (scanner.hasNextInt()) {
27                int value = scanner.nextInt();
28                if (value > 0) {
29                    return value;
30                }
31            }
32            System.out.println("Ошибка! Введите положительное число!");
33        }
34        } else {
35            System.out.println("Ошибка! Введите целое число!");
36        }
37    }
38 }

```

```

32 scanner.next(); //очищаем некорректный ввод
33     }
34 }
35 }
36 public static double[][] multiplyOptimized(double[][]
firstMatrix, double[][] secondMatrix) {
37     int rows1 = firstMatrix.length;
38     int cols1 = firstMatrix[0].length;
39     int cols2 = secondMatrix[0].length;
40     double[][] result = new double[rows1][cols2];
41     int blockSize = 64;
42     for (int blockI = 0; blockI < rows1; blockI += blockSize
) {
43         for (int blockJ = 0; blockJ < cols2; blockJ +=
blockSize) {
44             for (int blockK = 0; blockK < cols1; blockK +=
blockSize) {
45                 for (int i = blockI; i < Math.min(blockI +
blockSize, rows1); i++) {
46                     for (int k = blockK; k < Math.min(blockK
+ blockSize, cols1); k++) {
47                         for (int j = blockJ; j < Math.min(
blockJ + blockSize, cols2); j++)
48                             {
result[i][j] += firstMatrix[i][k
] * secondMatrix[k][j];
49                             }
50                     }
51                 }
52             }
53         }
54     }
55     return result;
56 }

57
58 public static void main(String[] args) {
59     Scanner scanner = new Scanner(System.in);
60     int firstRows = readPositiveInt(scanner, "Введите количе
ство строк для первой матрицы: ");
61     int firstColumns = readPositiveInt(scanner, "Введите кол
ичество столбцов для первой матрицы: ");
62     int secondRows = readPositiveInt(scanner, "Введите колич
ество строк для второй матрицы: ");
63     int secondColumns = readPositiveInt(scanner, "Введите ко
личество столбцов для второй матрицы: ");
64     if (firstColumns != secondRows) {
65         System.out.println("Ошибка! Матрицы такой размерност
и нельзя перемножить! Столбцов в первой матрице д

```

```

        олжно быть столько же, сколько строк во второй!")
        ;
66 scanner.close();
67 return;
68 }
69 double [][] matrix1 = new double[firstRows][firstColumns
70 ];
71 double [][] matrix2 = new double[secondRows][
72 secondColumns];
73 for (int i = 0; i < firstRows; i++) {
74     for (int j = 0; j < firstColumns; j++) {
75         matrix1[i][j] = Math.random();
76     }
77 }
78 for (int i = 0; i < secondRows; i++) {
79     for (int j = 0; j < secondColumns; j++) {
80         matrix2[i][j] = Math.random();
81     }
82 }
83 int runs = 10;
84 long totalTime = 0;
85 for (int run = 0; run < runs; run++) {
86     long startTime = System.nanoTime();
87     double [][] result = multiply(matrix1, matrix2);
88     long endTime = System.nanoTime();
89     long duration = (endTime - startTime) / 1_000_000;
90     totalTime += duration;
91     System.out.println((run + 1) + ": " + duration + " м
92     с");
93 }
94 System.out.println("Среднее время за 10 перемножений: "
95     + (totalTime / runs) + " мс");
96 totalTime = 0;
97 for (int run = 0; run < runs; run++) {
98     long startTime = System.nanoTime();
99     double [][] result = multiplyOptimized(matrix1,
100     matrix2);
101     long endTime = System.nanoTime();
102     long duration = (endTime - startTime) / 1_000_000;
103     totalTime += duration;
104     System.out.println((run + 1) + ": " + duration + " м
105     с");
106 }
107 System.out.println("Среднее время за 10 оптимизированных
108     перемножений: " + (totalTime / runs) + " мс");
109 scanner.close();
110 }
111 }

```

3.3 Список вопросов и ответов на них

Ниже представлен список из 80 вопросов, которые специалист по тестированию задавал программисту. После каждого вопроса приведен ответ. После некоторых ответов следуют замечания специалиста по тестированию.

Обращения к данным

1. Используются ли переменные с неинициализированными значениями?

Нет. Все локальные переменные инициализируются явно либо присваиванием перед использованием.

2. Выходят ли индексы за границы массива?

Нет.

- Во всех циклах границы заданы через `.length` или `Math.min(...)`.

- При индексации `firstMatrix[i][j]` гарантируется, что `i` и `j` в допустимых пределах.

3. Используются ли нецелочисленные индексы?

Нет. Все индексы — `int`.

4. Есть ли «висячие» ссылки?

Нет. Массивы не освобождаются вручную, ссылки на них живут только внутри методов и теряются после выхода.

5. Корректны ли атрибуты всех псевдонимов?

Вопрос не применим. В Java нет псевдонимов в смысле Fortran EQUIVALENCE.

6. Согласуются ли атрибуты структуры данных и физической записи?

Не применимо. Нет работы с физическими файлами записей фиксированной структуры.

7. Вычисляются ли адреса битовых строк? Передаются ли битовые строки в качестве аргументов?

Нет. Битовые строки отсутствуют.

8. Корректно ли используются атрибуты памяти, адресуемой с помощью ссылок и указателей?

Да. Ссылки на массивы передаются корректно, изменений в адресной арифметике нет.

9. Согласуются ли описания структуры данных в разных процедурах?

Да. Матрицы везде описываются как `double[][]`.

10. Ошибки завышения/занижения индекса на единицу?

Нет. Все индексы начинаются с 0 и идут до `< .length`.

11. Соблюдены ли правила наследования объектов?

Не применимо. Наследование не используется.

12. Правильно ли обработан случай, когда размер матрицы не кратен размеру блока?

Да, Math.min обеспечивает корректные границы последнего блока.

13. Могут ли остаться ссылки на объекты после выхода из области видимости?

Нет. Все ссылки локальны и уничтожаются при выходе из метода.

Вычисления

1. Участвуют ли в вычислениях неарифметические переменные?

Нет. Только double, int, long.

2. Выполняются ли вычисления с использованием данных разного типа?

Да, в допустимых пределах:

- int -> long (вычитание nanoTime() -> duration).
- int -> double при умножении firstMatrix[i][j] * secondMatrix[j][k] (расширение).
- Нет неявного преобразования в меньший тип.

3. Выполняются ли вычисления с переменными разной длины?

Не применимо. Java не имеет понятия «разная длина» для примитивов фиксированного размера.

4. Присваиваются ли переменным значения типа большего размера?

Нет. double получает double, int — int, long — long.

5. Возможны ли переполнение или потеря точности в промежуточных вычислениях?

Переполнение:

- duration — long, при текущих значениях переполнение невозможно.
- Индексы остаются в пределах int, переполнение индексов при $i += blockSize$ теоретически возможно при очень больших размерах матриц ($>2^{31}-1$), что нереалистично.

Потеря точности:

- double умножения — стандартно для IEEE 754.

6. Возможно ли деление на ноль?

Нет. Деления в коде нет.

7. Критичны ли погрешности из-за двоичного представления чисел?

Нет. Погрешности есть, но они не влияют на корректность алгоритма и сравнения не выполняются.

8. Не выходят ли значения переменных за логически обоснованные пределы?

Нет.

- rows, cols > 0.
- blockSize = 64 — фиксировано, не превышает размеры.
- run от 0 до 9.

9. Понятны ли приоритеты операций?

Да. Выражения простые, скобки не требуются.

10. Правильно ли используются результаты деления целых чисел?

Нет деления целых (кроме totalTime / runs — long / int -> long, это допустимо).

11. Есть ли операции с логическими значениями в арифметических выражениях?

Нет.

12., Есть ли риск случайного доступа к package-private членам из других пакетов?

Нет, класс в отдельном файле, всё public.

Описания данных

1. Все ли переменные объявлены?

Да.

2. Понятны ли последствия использования атрибутов по умолчанию?

Да.

- public — осознанно.
- static — утилитарный класс.
- package-private — у членов класса.

3. Правильно ли инициализированы массивы и строки?

Да. Матрицы заполняются Math.random(), строковые константы корректны.

4. Правильно ли определены размеры, типы и классы памяти?

Да. Размеры задаются во время выполнения через new double[...][...].

5. Сопласуется ли инициализация с классом памяти?

Да. Локальные переменные инициализированы перед использованием.

6. Имеются ли переменные с похожими именами?

Нет. firstRows, firstColumns, secondRows, secondColumns — имена различимы.

7. Есть ли сравнения с плавающей точкой на равенство?

Нет.

Сравнения

1. Есть ли попытки сравнения несопоставимых переменных?

Нет. Сравняются только int и int.

2. Есть ли попытки сравнения переменных разного типа?

Нет.

3. Корректно ли используются операторы сравнения?

Да. Только `==` и `!=` с числами, `>` с числом.

4. Корректно ли используются булевы выражения?

Да. В `if` и `while` используются простые условия.

5. Корректно ли используются результаты сравнения в булевых выражениях?

Да.

6. Корректно ли сравниваются дробные величины?

Нет сравнения дробных величин.

7. Понятны ли приоритеты операций?

Да.

8. Понятен ли способ разбора булевых выражений компилятором?

Не применимо.

9. Сравниваются ли объекты через `==` вместо `equals`?

Нет, объекты не сравниваются.

Передача управления

1. Может ли значение индекса в переключателе превысить число переходов?

Нет `switch`.

2. Будет ли завершен каждый цикл?

Да. Все циклы имеют счётчики, монотонно возрастающие к условию выхода.

3. Будет ли завершена программа?

Да. `main` завершается после закрытия `Scanner`.

4. Есть ли цикл, который может не выполниться из-за входных условий?

Да, и это корректно. В `readPositiveInt` цикл `while (true)` гарантированно выполняется, пока не вернёт значение. В блочном умножении `for (int blockI = 0; blockI < rows1; blockI += blockSize)` — если `rows1 = 0`, цикл не выполнится. Это корректно, так как размеры матриц положительные по входному контролю.

5. Корректны ли возможные погружения в цикл?

Да. Тройная вложенность циклов — стандарт для умножения матриц.

6. Ошибки диапазона, приводящие к отклонению количества итераций?

Нет. Все итерации вычислены корректно.

7. Соответствует ли каждому оператору `DO` свой оператор `END`?

Не применимо. Java использует .

8. Существуют ли точки ветвления, в которых не учтены все возможные условия?

Нет. В `readPositiveInt` учтены:

- не число
- отрицательное/ноль
- положительное

Все пути закрыты.

9. Есть ли ветвления, где пропущен `break`?

Нет.

Ввод-вывод

1. Правильно ли заданы атрибуты файлов?

Нет файлового ввода-вывода.

2. Корректны ли инструкции `OPEN`?

Не применимо.

3. Сходятся ли спецификации формата с инструкциями ввода-вывода?

Нет форматированного вывода.

4. Соответствует ли размер буфера размеру записи?

Не применимо.

5. Открываются ли файлы перед обращением к ним?

Не применимо.

6. Закрываются ли файлы после обращения к ним?

Да, `scanner.close()` — закрытие стандартного ввода, что допустимо.

7. Корректно ли обрабатываются признаки конца файла?

Не применимо.

8. Обрабатываются ли ошибки ввода-вывода?

Да. `hasNextInt()` — защита от неверного ввода.

9. Присутствуют ли в выходной информации орфографические или грамматические ошибки?

Нет. Текст на русском — корректен.

10. Есть ли риск несоответствия разделителей строк в разных ОС?

Нет, используется `System.out.println()`.

11. Закрывается ли `Scanner` при исключении?

Нет, если исключение произойдёт до `close()`, ресурс не закроется. Можно использовать `try-with-resources`.

Программные интерфейсы

1. Совпадает ли число формальных параметров с числом аргументов?

Да. `multiply(double[], double[])` вызывается с двумя аргументами.

2. Совпадают ли атрибуты параметров и аргументов?

Да. Оба — `double[]`.

3. Совпадают ли единицы измерения параметров и аргументов?

Не применимо. Физические единицы не используются.

4. Совпадает ли число аргументов, передаваемых вызываемым модулям?

Да.

5. Совпадают ли атрибуты аргументов с атрибутами ожидаемых параметров?

Да.

6. Совпадают ли единицы измерения?

Не применимо.

7. Правильно ли заданы количество, атрибуты и порядок следования аргументов при вызове встроенных функций?

Да. `Math.random()`, `System.nanoTime()`, `Math.min()` — корректно.

8. Имеются ли ссылки на параметры, не связанные с текущей точкой входа?

Нет.

9. Делаются ли в подпрограмме попытки изменить входные аргументы?

Нет. Матрицы не изменяются, результат — новый массив.

10. Согласуются ли определения глобальных переменных в разных модулях?

Нет глобальных переменных.

11. Передаются ли константы в качестве аргументов?

Да.

- 10 в цикле `for (int run = 0; run < runs; run++)`. - 1_000_000 при делении. - 64 — размер блока.

Это допустимо.

12. Правильно ли используются функции с переменным числом аргументов?

Нет таких вызовов.

13. Если бы аргументы изменялись, было бы это документировано?

Не применимо.

Другие проверки

1. Есть ли в таблице перекрестных ссылок переменные, на которые нет ссылок?

Не применимо. Нет таблицы перекрёстных ссылок.

2. Совпадает ли список атрибутов с тем, который следовало ожидать?

Да. Класс утилитарный, все методы public static.

3. Выдаются ли какие-нибудь предупреждения или информационные сообщения при компиляции?

Нет. Код компилируется без предупреждений (при стандартных настройках).

4. Осуществляется ли проверка корректности входных данных?

Да.

- readPositiveInt — проверка типа и диапазона.

- if (firstColumns != secondRows) — проверка совместимости размеров.

5. Нет ли пропущенных функций?

Нет. Реализовано ровно то, что требуется для задачи.

6. Есть ли подавленные предупреждения (@SuppressWarnings)?

Нет.

4 Код программы с внесенными изменениями

Результатом метода инспекции кода является список правок, которые необходимо внести в программу:

1) добавить проверку на пустые матрицы в методы умножения — защита от `ArrayIndexOutOfBoundsException` при доступе к `firstMatrix[0].length` для пустого массива;

2) вынести "магические числа" в константы — `RUNS_COUNT = 10`, `BLOCK_SIZE = 64`, `NANOS_TO_MILLIS = 1_000_000` для улучшения читаемости и сопровождения кода;

3) улучшить обработку ресурсов — использовать `try-with-resources` для автоматического закрытия `Scanner`;

4) улучшить читаемость — добавить комментарии к сложным блокам кода (блочное умножение, измерение производительности).

Измененный код программы представлен в листинге ниже.

Листинг 2. Модифицированная реализация

```
1 package testing1;
2
3 import java.util.Scanner;
4
5 /**
6  * Класс для умножения матриц с измерением производительности
7  * Стандартный и оптимизированный (блочный) алгоритмы
8  */
9 public class MatrixMult {
10     // Константы для улучшения читаемости и сопровождения кода
11     private static final int RUNS_COUNT = 10;           // Количество прогонов для измерения времени
12     private static final int BLOCK_SIZE = 64;           // Размер блока для оптимизированного умножения
13     private static final long NANOS_TO_MILLIS = 1_000_000; // Коэффициент перевода наносекунд в миллисекунды
14
15     /**
16      * Стандартное умножение матриц (тройной вложенный цикл)
17      * @param firstMatrix первая матрица (m x n)
18      * @param secondMatrix вторая матрица (n x p)
19      * @return матрица-результат (m x p)
20      * @throws IllegalArgumentException если матрицы пустые или несовместимые
21      */
22     public static double[][] multiply(double[][] firstMatrix,
23         double[][] secondMatrix) {
24         // Защита от пустых матриц
```

```

24     if (firstMatrix == null || secondMatrix == null) {
25         throw new IllegalArgumentException("Матрицы не должн
        ы быть null");
26     }
27     if (firstMatrix.length == 0 || secondMatrix.length == 0)
28     {
29         throw new IllegalArgumentException("Матрицы не должн
        ы быть пустыми");
30     }
31     if (firstMatrix[0].length == 0 || secondMatrix[0].length
        == 0) {
32         throw new IllegalArgumentException("Матрицы не должн
        ы иметь нулевую размерность");
33     }
34
35     int firstRows = firstMatrix.length;
36     int firstColumns = firstMatrix[0].length;
37     int secondRows = secondMatrix.length;
38     int secondColumns = secondMatrix[0].length;
39
40     // Проверка совместимости размеров для умножения
41     if (firstColumns != secondRows) {
42         throw new IllegalArgumentException("Несовместимые ра
        змеры матриц: " +
43             firstRows + "x" + firstColumns + " и " +
44             secondRows + "x" + secondColumns);
45     }
46
47     double[][] result = new double[firstRows][secondColumns
        ];
48
49     // Стандартный алгоритм умножения: i, j, k
50     for (int i = 0; i < firstRows; i++) {
51         for (int j = 0; j < firstColumns; j++) {
52             double temp = firstMatrix[i][j]; // Оптимизация:
53             // уменьшение количества обращений к памяти
54             for (int k = 0; k < secondColumns; k++) {
55                 result[i][k] += temp * secondMatrix[j][k];
56             }
57         }
58     }
59
60     return result;
61 }
62
63 /**
64  * Ввод положительного целого числа с консоли с проверкой ко
        ректности

```

```

62  * @param scanner объект Scanner для ввода
63  * @param message приглашение для ввода
64  * @return положительное целое число
65  */
66  public static int readPositiveInt(Scanner scanner, String
    message) {
67      while (true) {
68          System.out.print(message);
69          if (scanner.hasNextInt()) {
70              int value = scanner.nextInt();
71              if (value > 0) {
72                  return value;
73              } else {
74                  System.out.println("Ошибка! Введите положи-
                    тельное число!");
75              }
76          } else {
77              System.out.println("Ошибка! Введите целое число!
                    ");
78              scanner.next(); // очищаем некорректный ввод
79          }
80      }
81  }
82
83  /**
84   * Оптимизированное умножение матриц (блочный алгоритм)
85   * @param firstMatrix первая матрица (m x n)
86   * @param secondMatrix вторая матрица (n x p)
87   * @return матрица-результат (m x p)
88   * @throws IllegalArgumentException если матрицы пустые или
            несовместимые
89   */
90  public static double[][] multiplyOptimized(double[][]
    firstMatrix, double[][] secondMatrix) {
91      // Защита от пустых матриц
92      if (firstMatrix == null || secondMatrix == null) {
93          throw new IllegalArgumentException("Матрицы не должн
                    ы быть null");
94      }
95      if (firstMatrix.length == 0 || secondMatrix.length == 0)
          {
96          throw new IllegalArgumentException("Матрицы не должн
                    ы быть пустыми");
97      }
98      if (firstMatrix[0].length == 0 || secondMatrix[0].length
          == 0) {
99          throw new IllegalArgumentException("Матрицы не должн
                    ы иметь нулевую размерность");

```

```

100     }
101
102     int rows1 = firstMatrix.length;
103     int cols1 = firstMatrix[0].length;
104     int rows2 = secondMatrix.length;
105     int cols2 = secondMatrix[0].length;
106
107     // Проверка совместимости размеров для умножения
108     if (cols1 != rows2) {
109         throw new IllegalArgumentException("Несовместимые ра
110             змеры матриц: " +
111                 rows1 + "x" + cols1 + " и " + rows2 + "x" +
112                 cols2);
113     }
114
115     double [][] result = new double[rows1][cols2];
116
117     // Блочное умножение для улучшения локальности данных (о
118     птимизация кэша)
119     // Разбиваем матрицы на блоки размером BLOCK_SIZE x
120     BLOCK_SIZE
121
122     // Внешние циклы по блокам
123     for (int blockI = 0; blockI < rows1; blockI +=
124         BLOCK_SIZE) {
125         for (int blockJ = 0; blockJ < cols2; blockJ +=
126             BLOCK_SIZE) {
127             for (int blockK = 0; blockK < cols1; blockK +=
128                 BLOCK_SIZE) {
129
130                 // Внутренние циклы по элементам внутри теку
131                 щего блока
132                 // Используем Math.min для обработки последн
133                 его блока, который может быть неполным
134                 int iMax = Math.min(blockI + BLOCK_SIZE,
135                     rows1);
136                 int kMax = Math.min(blockK + BLOCK_SIZE,
137                     cols1);
138                 int jMax = Math.min(blockJ + BLOCK_SIZE,
139                     cols2);
140
141                 for (int i = blockI; i < iMax; i++) {
142                     for (int k = blockK; k < kMax; k++) {
143                         double temp = firstMatrix[i][k]; //
144                         Оптимизация: уменьшение количеств
145                         а обращений к памяти
146                         for (int j = blockJ; j < jMax; j++)
147                             {

```

```

133         result[i][j] += temp *
            secondMatrix[k][j];
134     }
135 }
136 }
137 }
138 }
139 }
140
141     return result;
142 }
143
144 /**
145  * Точка входа в программу
146  * @param args аргументы командной строки (не используются)
147  */
148 public static void main(String[] args) {
149     // Использование try-with-resources для автоматического
        закрытия Scanner
150     try (Scanner scanner = new Scanner(System.in)) {
151
152         // Ввод размеров матриц с проверкой на положительные
            числа
153         int firstRows = readPositiveInt(scanner, "Введите ко
                личество строк для первой матрицы: ");
154         int firstColumns = readPositiveInt(scanner, "Введите
                количество столбцов для первой матрицы: ");
155         int secondRows = readPositiveInt(scanner, "Введите к
                оличество строк для второй матрицы: ");
156         int secondColumns = readPositiveInt(scanner, "Введит
                е количество столбцов для второй матрицы: ");
157
158         // Проверка на возможность перемножения
159         if (firstColumns != secondRows) {
160             System.out.println("Ошибка! Матрицы такой размер
                ности нельзя перемножить! " +
161                 "Столбцов в первой матрице должно быть с
                только же, сколько строк во второй!")
                ;
162             return; // Scanner закроется автоматически благо
                даря try-with-resources
163         }
164
165         // Создание матриц
166         double[][] matrix1 = new double[firstRows][
                firstColumns];
167         double[][] matrix2 = new double[secondRows][
                secondColumns];

```



```

168 // Заполнение матриц случайными значениями
169 for (int i = 0; i < firstRows; i++) {
170     for (int j = 0; j < firstColumns; j++) {
171         matrix1[i][j] = Math.random();
172     }
173 }
174
175 for (int i = 0; i < secondRows; i++) {
176     for (int j = 0; j < secondColumns; j++) {
177         matrix2[i][j] = Math.random();
178     }
179 }
180
181 System.out.println("\n— Измерение производительности стандартного умножения —");
182 // Измерение времени для стандартного умножения
183 long totalTime = 0;
184 for (int run = 0; run < RUNS_COUNT; run++) {
185     long startTime = System.nanoTime();
186     double[][] result = multiply(matrix1, matrix2);
187     long endTime = System.nanoTime();
188     long duration = (endTime - startTime) /
189         NANOS_TO_MILLIS;
190     totalTime += duration;
191     System.out.println((run + 1) + ": " + duration +
192         " мс");
193 }
194 System.out.println("Среднее время за " + RUNS_COUNT
195     + " перемножений: " +
196     (totalTime / RUNS_COUNT) + " мс");
197
198 System.out.println("\n— Измерение производительности оптимизированного умножения —");
199 // Измерение времени для оптимизированного умножения
200 totalTime = 0;
201 for (int run = 0; run < RUNS_COUNT; run++) {
202     long startTime = System.nanoTime();
203     double[][] result = multiplyOptimized(matrix1,
204         matrix2);
205     long endTime = System.nanoTime();
206     long duration = (endTime - startTime) /
207         NANOS_TO_MILLIS;
208     totalTime += duration;
209     System.out.println((run + 1) + ": " + duration +
210         " мс");
211 }
212 System.out.println("Среднее время за " + RUNS_COUNT
213     + " оптимизированных перемножений: " +

```

```
207         (totalTime / RUNS_COUNT) + " мс");
208
209     } // Scanner автоматически закрывается здесь благодаря
        try-with-resources
210 }
211 }
```

5 Обобщение работы

В ходе работы в программе были выявлены следующие недостатки: отсутствие проверки на пустые матрицы — исходная версия программы обращалась к `firstMatrix[0].length` без проверки, что могло привести к `ArrayIndexOutOfBoundsException` при передаче пустого массива; использование "магических чисел" — в коде использовались числовые значения 10, 64, 1_000_000 без пояснения их смысла, что затрудняло понимание и сопровождение кода; небезопасная работа с ресурсами — `Scanner` закрывался только в конце метода `main`, но при возникновении исключения до вызова `close()` ресурс оставался открытым; недостаточная документированность — сложные алгоритмы (блочное умножение) не содержали комментариев, поясняющих принцип работы и оптимизации; отсутствие проверки совместимости в методах умножения — проверка выполнялась только в `main`, но сами методы `multiply` и `multiplyOptimized` могли быть вызваны с некорректными размерами.

Была добавлена защита от пустых матриц и `null`-значений в методы умножения, вынесены "магические числа" в константы с понятными именами, реализовано автоматическое закрытие `Scanner` через `try-with-resources`, добавлены подробные комментарии к алгоритмам и сложным блокам кода, улучшена проверка совместимости размеров с информативными сообщениями об ошибках, добавлена микрооптимизация с временной переменной `temp` для уменьшения количества обращений к памяти.

Код в целом соответствует стандартам оформления, что делает его удобным для чтения и анализа. Внесенные изменения повысили надежность программы (защита от исключений), улучшили читаемость (константы, комментарии) и обеспечили корректное управление ресурсами.

Получен опыт проведения ручного тестирования методом инспекции кода, совместного разбора кода в группе, а также формулирования ответов на вопросы о реализации программы.

Заключение

В ходе выполнения данной лабораторной работы был изучен метод ручного тестирования — инспекция кода. Было проведено инспекционное собрание, на котором участвовали секретарь, программист и специалист по тестированию. По итогу собрания был составлен протокол заседания, далее программистом был проведен анализ составленного протокола и внесены необходимые изменения в код программы.

В результате выполненной работы можно сделать вывод, что методы ручного тестирования, в частности инспекция кода, являются эффективными для выявления ошибок в логике программы, а также для выявления несоответствий требованиям. Особенно ценным оказалось выявление скрытых дефектов, таких как потенциальные исключения при пустых массивах, которые сложно обнаружить при обычном тестировании.

В результате работы удалось улучшить читаемость кода, добавить поясняющие комментарии, обеспечить надежную обработку ресурсов и исправить потенциальные ошибки. Код стал более безопасным и поддерживаемым.

Список использованной литературы

- [1] Майерс, Г. Искусство тестирования программ. – Санкт-Петербург: Диалектика, 2012 г.